Assignment 3 – Identification of railway track events.

Participants

Tor Suneson. All by myself.

Source code available at https://github.com/torsuneson/Assignment3_public

AI Statement

I have use CoPilot from Microsoft to discuss with correct code when I have found it impossible to understand what was wrong with it, AI Spots the error immediately.

I also used VSCodium with Pyrefly (0.52.0) to auto complete when I started to write a segment.

Error handling, params and complex parts in functions it mostly done with the helper that "suggest" a piece of code that I have accepted. *I have not written all the complex code*, much of the complex parts are from stack overflow, with corrections from AI and the Pyrefly plugin.

Since this is not a programming course, and that the lab-instructor stated it was OK to use AI, that's how I have approached analysing the data.

Index

Assignment 3 – Identification of railway track events. 1
Participants 1
AI Statement 1
Background 3
Dataset: 3
Instructions for assignment 3
Note on feature sets 4
Data processing, Grade 3 4
Unsupervised Learning (Grade 4) 6
Dataset splitting 6
Support Vector Machine 8
Accuracy / Test error 8
Consistency of the results 8
The ability for generalization 8
Feature selection algorithms (Grade 5) 9
Normalization! 9
Filter methods 10
Wrapper methods 14
Multiclass 14
Binarization 15
Regularization strength 15
Embedded methods 17
Grade 5 part 2, implementation 18
Reference literature 19

Background

Switches are an essential, safety-critical part of railway infrastructure. Compared to open tracks, their complex geometry results in increased dynamic loading on the track superstructure caused by passing trains, leading to higher maintenance costs. To improve efficiency, condition monitoring methods specific to railway switches are necessary. A common approach to monitoring the track superstructure is to measure the acceleration caused by vehicle-track interaction. Local interruptions in the wheel-rail contact, caused by factors such as defects or track discontinuities, appear in the data as transient impact events.

In this assignment, these transient events are investigated using an ***experimental setup of a railway switch equipped with track-side acceleration sensors.*** *The analysis uses frequency and waveform data to understand the origins of these impact events and their implications for monitoring local track discontinuities and defects with wayside acceleration sensors in practice [Lab assignment].*

Dataset:

The data comprises three files, each containing extracted features from 1D acceleration sensor signals. These features include:

Statistical Features: *mean, std, max, min, range, skewness, kurtosis, rms, crest_factor, variance, zero_crossings.*

Frequency Features: *dominant_freq, spectral_energy, spectral_centroid, spectral_bandwidth, spectral_flatness.*

Each file includes a column labelled event, which serves as the ground truth for the presence of transient events.

Instructions for assignment

In assignment 3 have three levels of the assignment.

1. For Grade 3, Data processing of the three given files is required.
    a. Data preprocessing (Load, combine, remove and replace normal events with 0 and others with 1)
    b. Data Transformation (Normalize the data)
2. For Grade 4, ML evaluation.
    a. Split the data in training and testing sets in a 80/20 ratio
    b. Cross-validation (CV), by k-fold (5-fold) on the training set and evaluate the model stability.
    c. Compare between the 80/20 train-test split and the k-fold cross validation using SVM (Support Vector Machine). Discuss the difference in accuracy, consistency in results and generalization ability.
3. For Grade 5.
    a. Research filter methods, Wrapper methods, embedded methods
    b. Implement at least four (4) feature selection algorithms in this project and apply them to the dataset.

Note on feature sets

Generally, have I not considered the different properties of the dataset when analyzing the data.

Statistical features, or time-domain features, tend to be dominant because the variance is much larger than with the frequency-domain features.

So, the plots are generally done on statistical features since they are the ones that get the best scores when tested. But the generally the complete dataset with both statistical and frequency have been tested when the numeric scores have been acquired in the terminal window, and that's how it's presented in the report.

Data processing, Grade 3

In file `Assignment 3 Grade 3.py` are all data processing done.

File names are stored in an array so they can be easily changed, modified and switched to another set of files of any number.

```
# Filnamnen
files = [
    "Trail1_extracted_features_acceleration_m1ai1.csv",
    "Trail2_extracted_features_acceleration_m1ai1.csv",
    "Trail3_extracted_features_acceleration_m2ai0.csv",
]
```

For easy access and interchangeability, the columns that shall be dropped are stored in an array.

```
# Kolumner som ska tas bort om de finns, enligt instruktion
COLS_TO_DROP = ["start_time", "axle", "cluster", "tsne_1", "tsne_2"]
```

Then some support functions are added.

`load_and_combine_csv(file_list)` – a support function that imports and combines files into one file.

`drop_columns_safely(df, cols)` – a support function to drop any specified column sin a dataframe

`to_binary_event(series)` – a support function that maps Normal -> 0 and other events to 1.

`def normalize_and_save( input_csv, output_csv, exclude_cols, method)` – a support function that imports a CSV file (preferably the combined) an output a

normalized version of the file, has two modes as mentioned in the lab assignment. zscore which is the *standardized* and the *minmax*.

The main program that does the data processing execute these steps.

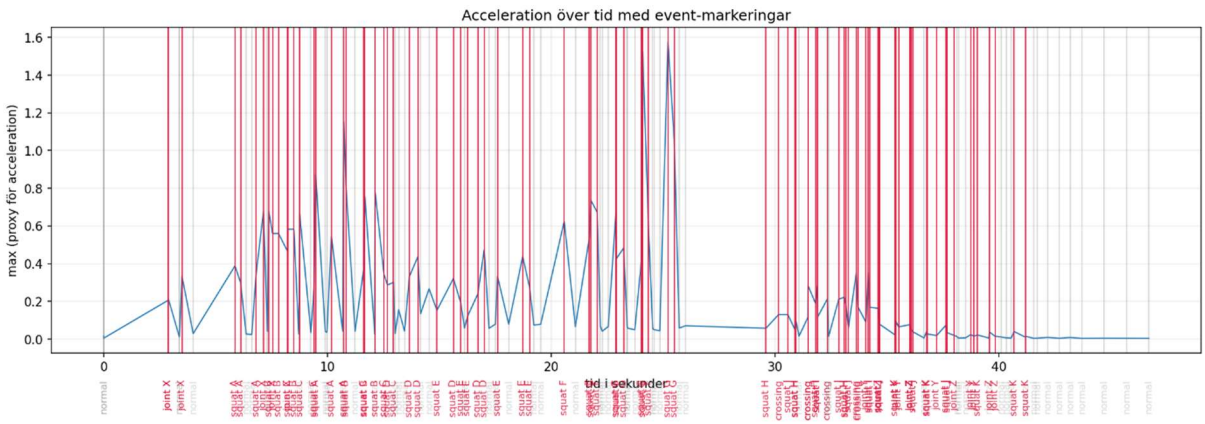
1. `load_and_combine_csv`
   Input the file list that shall be combined, output is the dataframe for further processing
2. `drop_columns_safley`, input is the dataframe, and columns to drop.
3. Test if the files have "EVENTS" to be able to process the data
4. Create a new column named "label" to store the binary events. `to_binary_event`
5. Save the dataframe with combined files and binary mapped events
6. Normalize the data, `normalize_and_save`, default using standardization. Labels that are excluded from normalization are, `label` (new, with binary events), `start_time` (if not excluded in the COLS_TO_DROP), and `event`. Since these columns have meta data and not measurements.

After the program has run two files have been created.

`combined_dataset.csv`

`combined_dataset_normalized.csv`

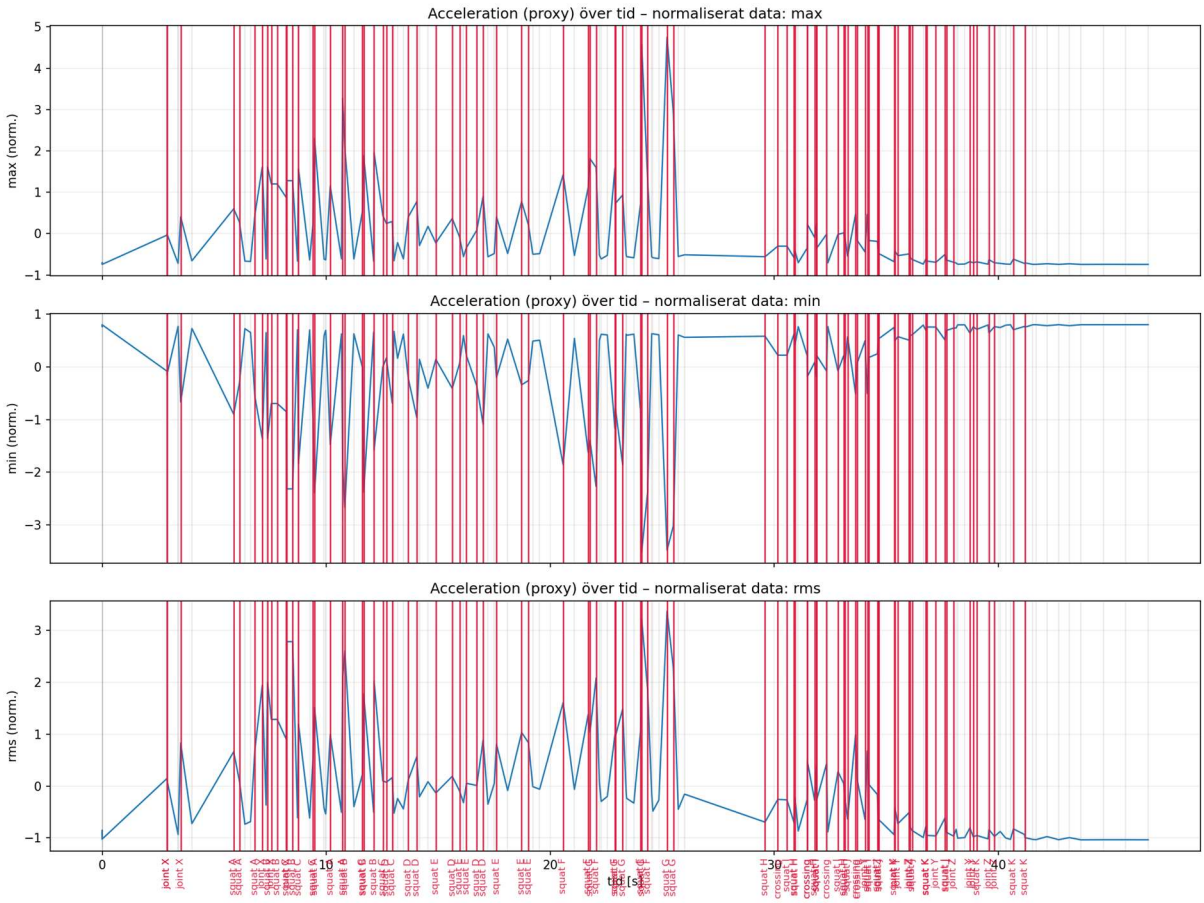
To be able to plot and see the result in these files, column `start_time`, needed to be included in in the data frame. That's why the support functions above have the ability to save the column.



Plotting the combined data without normalization max value are not within standardized range. The events have been binary mapped into normal (0) and others (1).

Plotting the normalized data done with standardization (z-score) the average is 0 and the standard deviation is 1. Standardization is the normalization we want if we are going to use SVM (Support Vector Machine) later.

SVM is considered to be effective in high-dimensional spaces and also said to be memory efficient. [7]



Unsupervised Learning (Grade 4)

All processning for grade 4 are done in `Assignment 3 Grade 4.py`.

Dataset splitting

To be able to train the Support Vector Machine (SVM) [9, ch 12] we need to split the data set. This can either be done in the 80/20 format where 80% of the data is used for training and 20% is used for validation. The other way to split the data is in even parts like the cross validation [9, ch 7.10] or k-fold. With the fold split the data is used the same number of times (k) as the split to train the model. The performance of the model it then the average performance for all the training sessions.

So, 80/20 split is generally considered a faster method, since it only runs once to train the ML but also less precise since result can vary with how data is split. All the data points are also not used for training the model since 20% is only used for validation

The cross validation is generally considered a much more stable method since all the data points are tested. But it has higher cost for reaching the result since you must run the training k number of times that the data was split in.

In summary, 80/20 have lower cost to get the result and can give a quick baseline, the cross validation or k-fold (5-fold) has a higher cost for calculating the result but in the other hand it gives better results at validation[9].

Several support functions have been created to manage the different split scenarios and the SVM training.

`split_80_20` Splits the data frame in an 80%, and 20% portion and returns the result

`make_kfold` Use `StratifiedKFold` from `sklearn.model_selection` to define set

`train_svm_80_20` Trains the SVM on the 80/20 split of the dataset

`train_svm_kfold` Trains the SVM on the k-fold dataset

C and gamma params loop to adjust the best fit and the distance that affect the decision point

`compute_errors_80_20` Compute the test and training error for 80/20 split

`compute_errors_kfold` Compute the test and training error for k-fold.

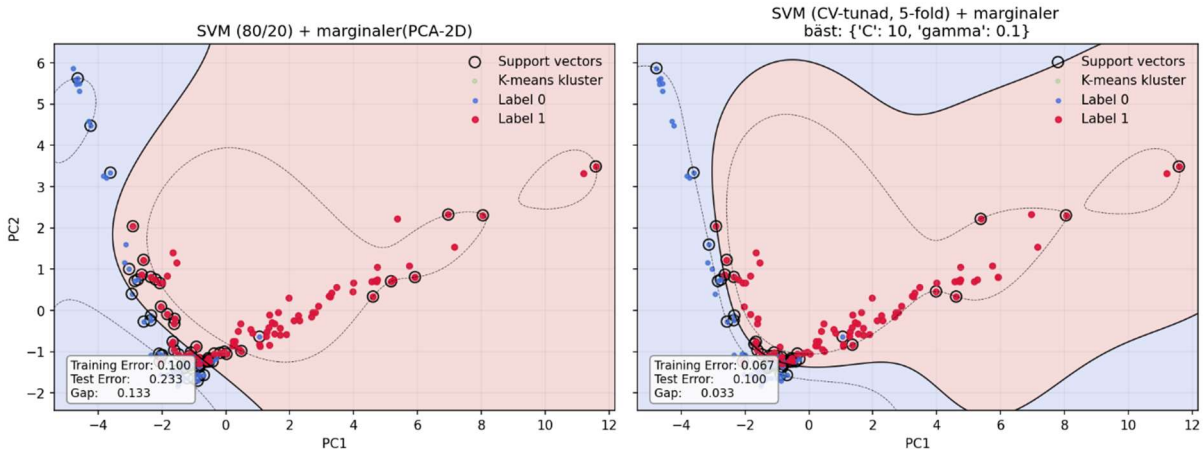
Another support function that shows the margin lines for the SVM have been added (as shown in lab introduction material).

`svm_margin_lines,` Will calculate the margin and the support vectors of the SVM, this will visually show the accuracy of the trained model.

Support Vector Machine

In this lab exercise two Support Vector Machines where trained using two different validation strategies. 80/20 split and k-fold (5-fold) cross-validation. The purpose of the comparison is to evaluate accuracy, consistency and the generalization ability.

Accuracy / Test error



For 80/20 the model was trained on 80% of the data and evaluated on the remaining 20%. This method gave a quick and reasonable evaluation of the model's performance; the results is dependent on the split of the data. Depending on which events that are in the training data and the test data the results could vary.

Using the k-fold cross-validation the model was trained 5 times (5-fold), where each part was used as test data once. The reported accuracy was based on the average of the fold-results. This gives a more robust and reliable estimation of the error, since each datapoint contributed to both training and testing.

Consistency of the results

There is a clear difference between the observed consistency in the methods. The 80/20 split only gave one test result and is thereby sensitive to random variation in the data split. Small changes in the split could lead to noticeable differences in test error.

Cross-validation, on the other hand, showed considerably higher stability since variation between the fold results where low. This suggests that the model's performance is not dependent on a single favourable, or unfavourable, split. It will better reflect the whole data set.

The ability for generalization

The ability to generalize is determined by comparing training error with test error, the gap between the. For the 80/20 model it was observed that there is a larger gap between the training and the test error. This indicates that risk for overfitting or insufficient representation if the test data.

For Cross-validation model the difference between training error and test error was generally lower. This indicates better generalization ability for the data. Cross-validation gives better ability to generalize and is more suitable when model evaluation is important [9, Ch 7.10.1].

Feature selection algorithms (Grade 5)

Normalization!

As I worked with the different steps in this assignment I learned more and more about the different techniques to process data. The grade 3 normalization technique was fetch from stackoverflow and modified, and when I moved into filters and wrappers I found out that `sklearn.preprocessing` has a lot of functions that do scaling and normalization.

Since I already did the Grade 3 implementation, I don't want to redo them, but for filters and wrappers I will use the more efficient `sklearn StandardScaler` for most methods and `MaxMinscaler` instead for the Chi2 algorithm.

Quick check on the algorithm characteristics and if they require normalization.

Method	**Does it need normalized data?**	**Why?**
Filters	Yes	Scales will affect the ranking, Chi2 need `minmax` scaling, the other filters need `Standardization`
Wrappers	Yes	SVM is scale sensitive
Embedded L1	Yes	L1-punishment is scale sensitive
Embedded tree	No	Trees are scale proof
SVM Training + CV	Yes	Standardized normalization needed.
PCA visualization	Yes	PCA is built on variance.

Filter methods

There are several filter methos that can be tried on the data set, and kernels, an array of filter (already used in `svm_margin lines`).

Common Filter techniques are: [Source]

- **Information Gain:** Measures reduction in entropy when a feature is used.
- **Chi-square test:** Checks the relationship between categorical features.
- **Fisher's Score:** Ranks features based on class separability.
- **Pearson's Correlation Coefficient:** Measures linear relationship between two continuous variables.
- **Variance Threshold:** Removes features with very low variance.
- **Mean Absolute Difference:** Similar to variance threshold but uses absolute differences.
- **Dispersion ratio:** Ratio of arithmetic mean to geometric mean; higher values indicate useful features.

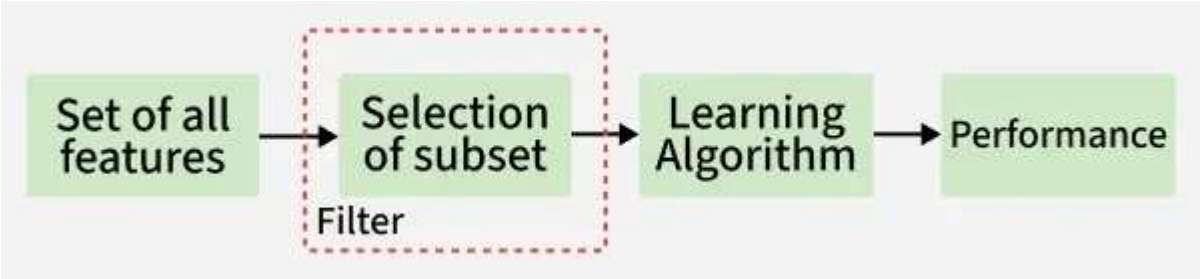


Figure 1: Filter methods (borrowed image)

Filters are fast and efficient and computational inexpensive, making them ideal for large datasets. Filters are relatively easy to implement and often built into machine learning libraries. Filters are a versatile tool since they can be used with any type of machine learning model.

A support file that tests and rank features based on filter top score. The filer file is named `feature_filters.py` and executed from `Assignment_3_Grade_5.py`.

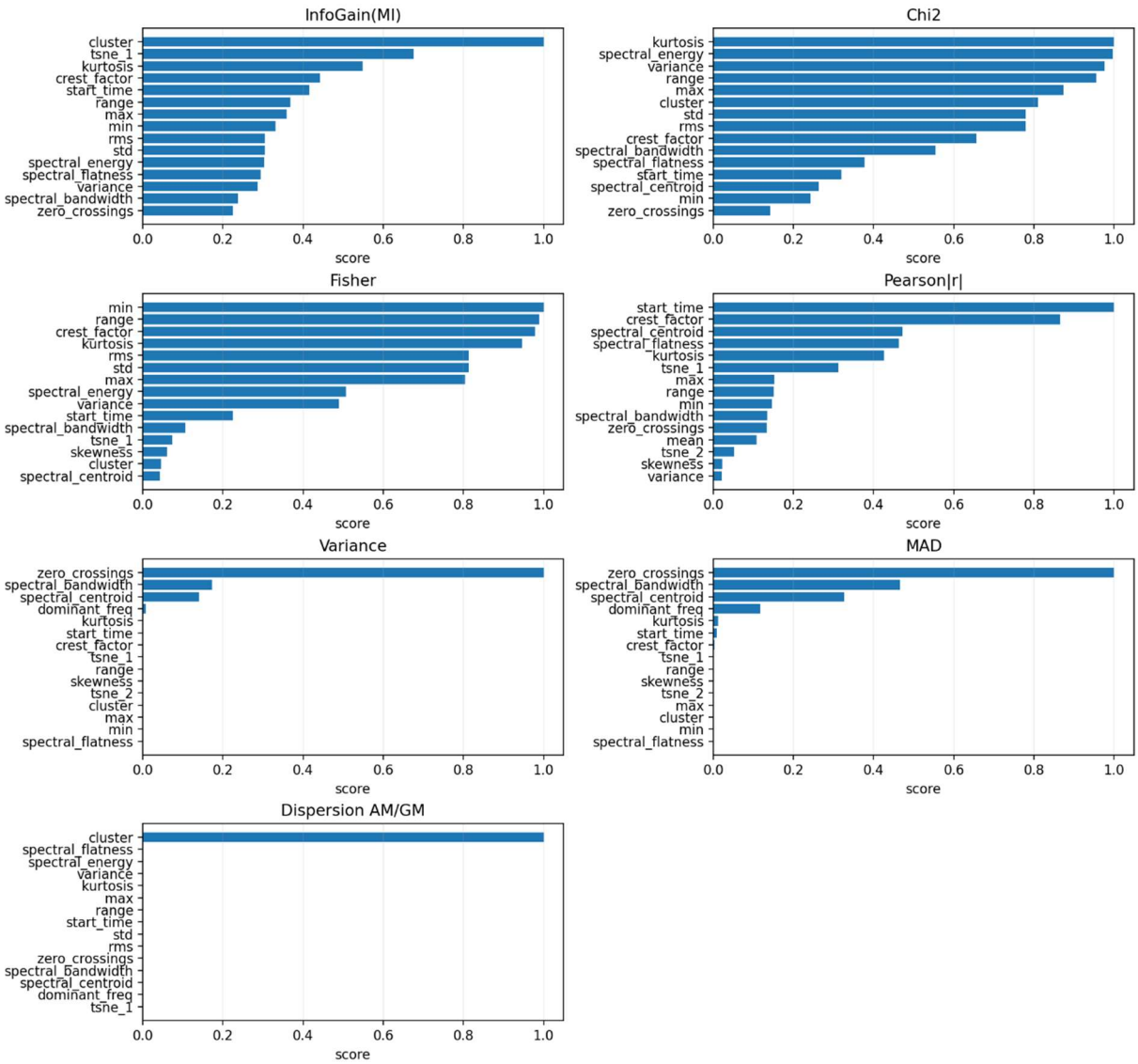


Figure 2: TopK resutlts of filters without binary classification

The Pearson correlation uses absolute values for ranking since both positive and negative linear relationships indicate informative features.

After discovering that wrapper classes best work with binary data (see below

Wrapper methods), I also investigated the correct properties for the filter functions.

- **Pearson correlation**, measure linear correlation and with binary data it becomes *point-biseral correlation*. This is a well-known measurement for continues feature and binary classification
- **Fisher score**, is explicit defined for two classes, with two classes Fisher gives correct results, with more classes generalization is needed
- **Chi-square**, measure dependency between feature and class, with two classes there will be higher counts and more stable test.
- **Information Gain (Mutual information),** MI works with more classes but requires more data then. With 150 samples and 16 classes the MI becomes noisy.

Information from log.

```
Class distribution (0=normal,1=event): [60, 90]

Unique classes in y: [0 1]

Number of classes: 2

Number of samples: 150
```

	Features																		
Filter	mean	std	max	min	range	skewness	kurtosis	rms	crest_factor	variance	zero_crossings	dominant_freq	spectral_energy	spectral_centroid	spectral_bandwidth	spectral_flatness	event	event_str	label
Infogain(MI)		0,454	0,526	0,441	0,510		1,000	0,454	0,904		0,450								
Chi2		0,497	0,596		0,672		1,000	0,497	0,998	0,477						0,692			
Fisher		0,218	0,241	0,272	0,264		0,441	0,218	1,000	0,136									
Pearson|r |		0,631	0,655	0,686	0,678		0,808	0,631	1,000	0,520									
Variance					0,000		0,000		0,000		1,000	0,008		0,140	0,173				
MAD					0,000	0,000	0,013		0,003		1,000	0,117		0,328	0,465				
Dispersion		0,098	0,242		0,231		0,360	0,098		0,710			0,716			1,000			

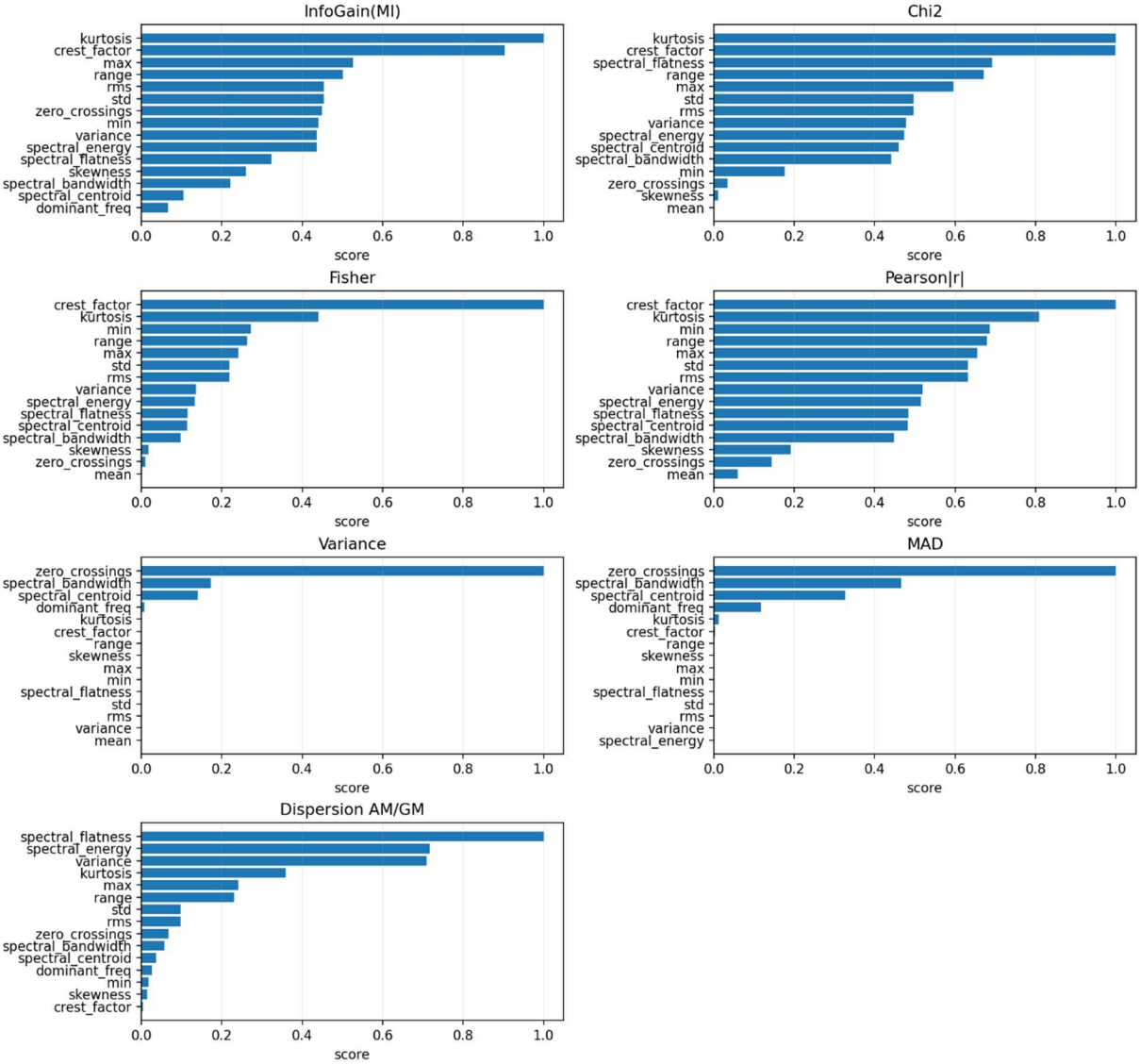
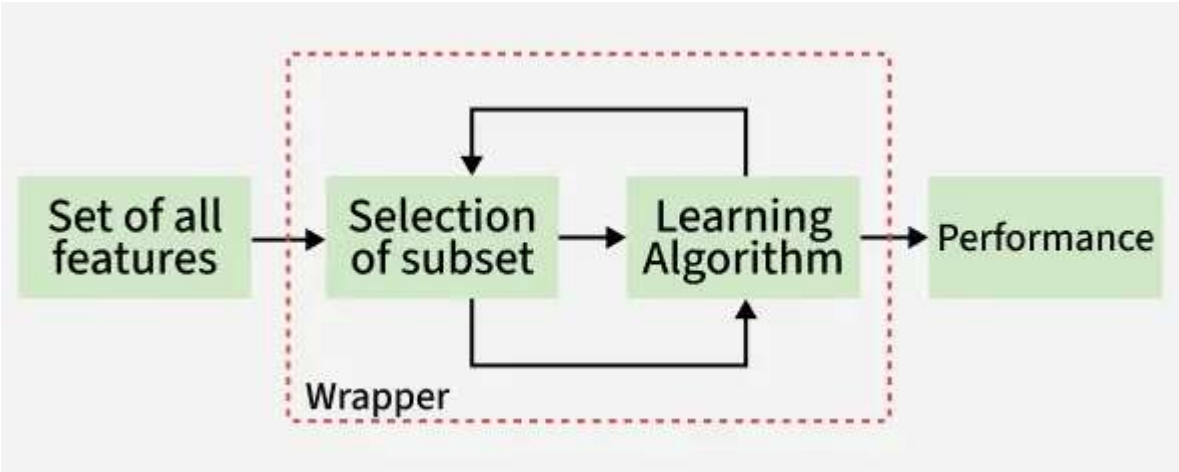


Figure 3: TopK results with binary classification, Correct way of using filters

Wrapper methods

According to GeekforGeek web page, wrappers are referred to as greedy algorithms that train algorithm.



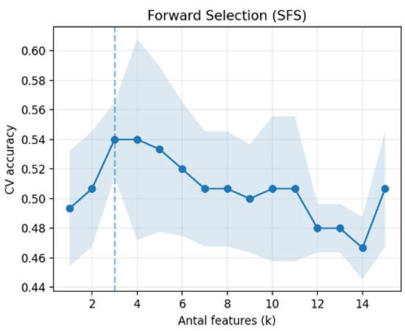
They calculate a subset of features by iterating over the feature set by removing and adding features from the original set. Stopping criteria for selecting the best subset are usually pre-defined by the person training the model. When performance is decreasing or a set number of features have been achieved the stopping criteria is met.

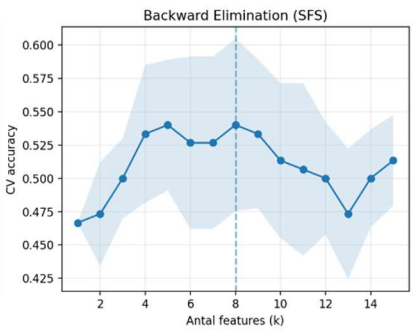
Common Wrapper Techniques [source]

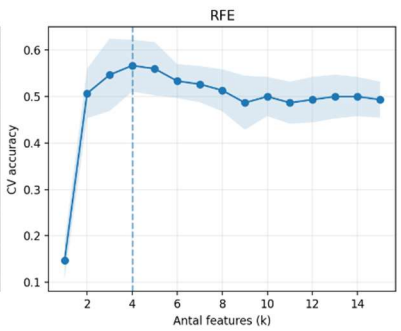
- **Forward Selection:** Start with no features and add one at a time based on improvement.
- **Backward Elimination:** Start with all features and remove the least useful ones.
- **Recursive Feature Elimination (RFE):** Removes the least important features step by step.

Multiclass

The import functions in data processing from Grade 3 exercise was used to fetch dataset and test the three wrapper methods described at Geeg4Geek. I get some strange errors and strange graphs looking when viewing the results.

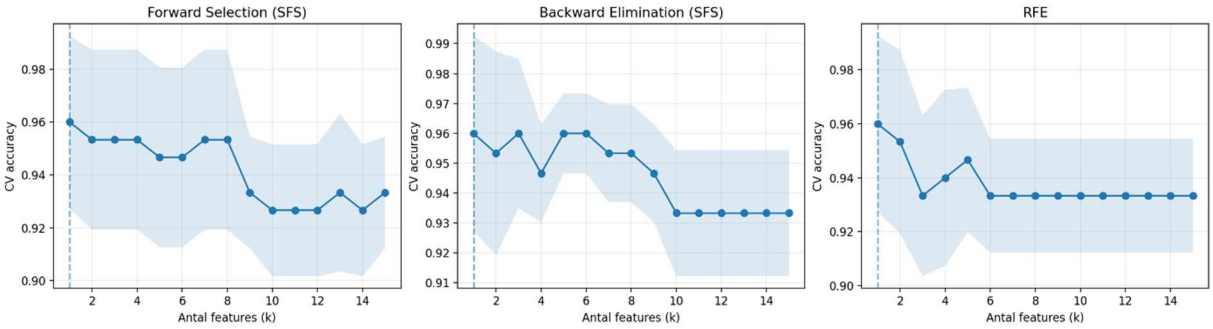






Binarization

During wrapper-based feature selection, warnings indicated that the target variable contained many unique values. This revealed that the original label was not properly encoded as a classification target. Trying binarizing of the event variable wrapper methods produced stable and interpretable performance curves.



The original event variable contained multiple numeric values and was therefore not directly suitable for classification. To enable wrapper-based feature selection, the event signal was converted into a binary target using the binary mapper from grade 3 (event / no-event).
This transformation ensured a well-defined classification task and stable cross-validation scores. The resulting class distribution (60 normal / 90 event) enabled stable cross-validation and meaningful wrapper performance curves.

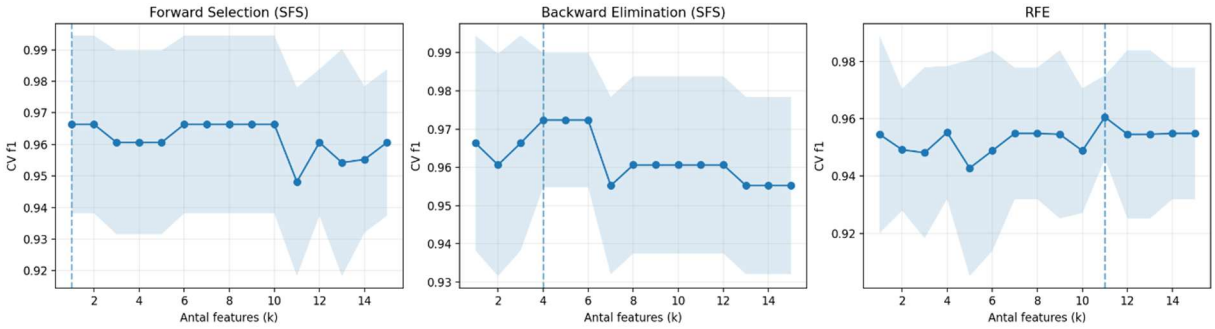
All wrapper methods consistently selected the same feature subset. This indicates that a single feature (crest factor) dominates the classification performance and is sufficient to separate normal and event samples. The agreement between forward selection, backward elimination and RFE suggests a stable and well-conditioned feature selection problem.

Best selections:	*k*	*score*	*Feature*
Forward Selection	1	0,96	Crest_factor
Backward Elim.	1	0,96	Crest_factor
Backward Elim.	1	0,96	Crest_factor

In the image above we can see cross-validation accuracy as a function of the number of selected features. The shaded area represents the variability across cross-validation folds.

Regularization strength

Let's see if we can improve that by setting regularization strength to 10 instead of default 1, as we saw the best results from the k-fold at C=10.



Best selections:	*k*	*score*	*Feature*
Forward Selection	1	0,966	Crest_factor
Backward Elim.	4	0,972	Crest_factor, spectral_energy, spectral_bandwidth, spectral_flatness
Backward Elim.	11	0,961	max, min, kurtosis, crest_factor, variance, zero_crossings, dominant_freq, spectral_energy, spectral_centroid, spectral_flatness

C is the inverse of regularization strength and means that lower C have stronger regularization and higher C give weaker regularization. By increasing C the L2 punishment becomes weaker and the improves the cross-validation performance. This indicates that the default regularization strength was overly restrictive and that the data contains strongly discriminative features that benefit from weaker regularization.

Embedded methods

Embedded methods are a cross between filters and wrappers; they select features during the training.

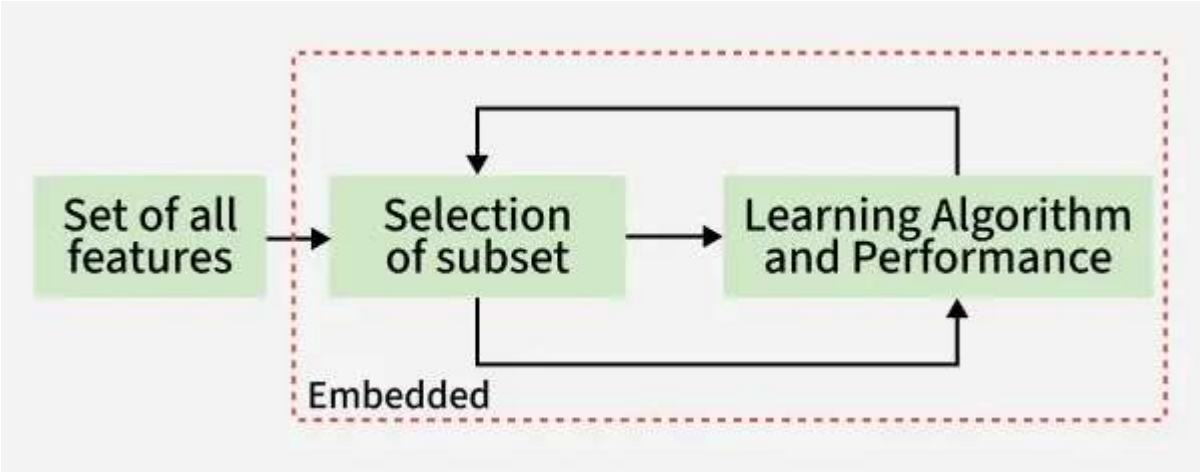


Figure 4 : Embedded methods

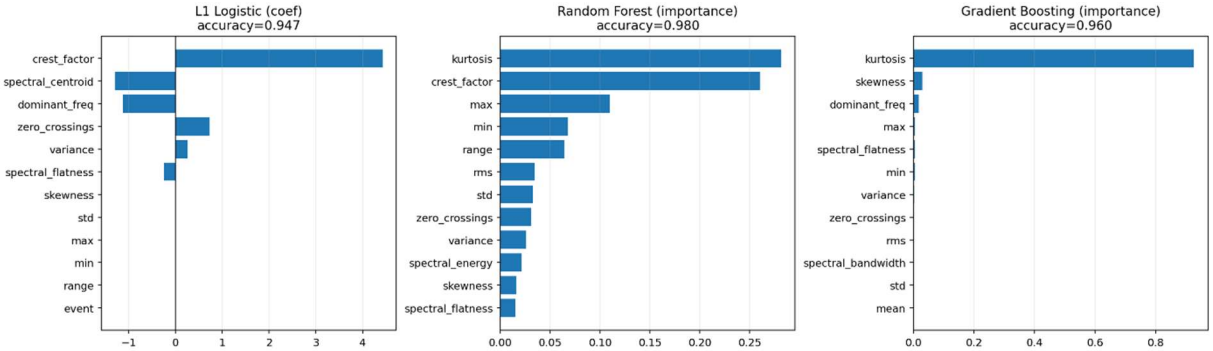


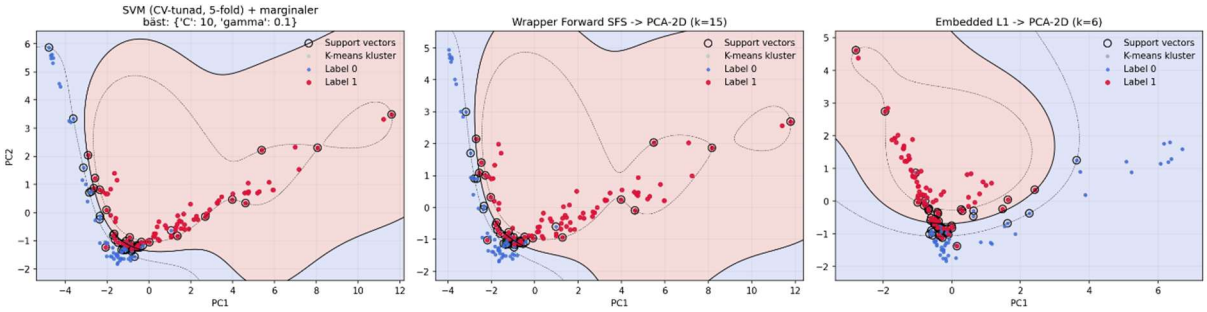
Figure 5: Embedded methods using binary data

While wrapper methods based on logistic regression selected `crest_factor` as the single most informative feature, tree-based embedded methods consistently ranked `kurtosis` as the most important feature.
This difference arises from the underlying model assumptions: linear models favour features with strong monotonic relationships, whereas tree-based models favour features that enable effective threshold-based splits.
Both features capture impulsive behaviour in the acceleration signal and are therefore physically meaningful indicators of transient track events.

Embedded methods (CV):	*accuracy*	*selected*
L1 (LogReg/Lasso)	0.947	crest_factor
Random Forest	0.980	kurtosis
Gradient Boosting	0.960	kurtosis

Implementation (Grade 5, part 2)

The last part of the assignment, as I understand it, was to implement filters, wrappers and embedded-methods and compare with previous results. It says to implement 4 feature selection algorithms. I have run out of time and need to move on. What I have done is reused 2 of the methods that was evaluated. The wrapper `forward_sfs` and the embedded method `L1` Since both performed well .



Using the (kind of) same plot as in Grade 4 we can see that the wrapper `forward_sfs` have found an alternative feature (but still many features left) that give about the same performance as the `SVM (CV-tuned)` have in from the original comparison that use all features.

However, the `L1` gives a simpler, more stable model with less overfitting, the decision boundary is more symmetrical and wider.

I'm sure there are more analysis that can be performed on the data but for now I will have to restrain myself.

Reference literature

1. **Paul Barry** - Headfirst Python. 3rd ed. *A Learner's Guide to the Fundamentals of Python Programming*-O'Reilly (2023), ISBN: 978-1-492-05129-9
2. **Ben Auffarth** - *Artificial Intelligence with Python Cookbook, Proven recipes for applying AI algorithms and deep learning techniques using TensorFlow 2.x and PyTorch 1.6-Packt Publishing* (2020), ISBN 978-1-78913-396-7
3. **Denis Rothman** - *Transformers for Natural Language Processing 2nd ed. Build, train, and fine-tune deep neural network architectures for NLP with Python, PyTorch, TensorFlow, BERT, and GPT-3*, ISBN 978-1-80324-733-5
4. **Aman Kedia, Mayank Rasu** - *Hands-On Python Natural Language Processing, Explore tools and techniques to analyze and process text with a view to building real-world NLP applications*, Packt Publishing Ltd (2020), ISBN 978-1-83898-959-0
5. **Sayan Mukhopadhyay** - *Advanced Data Analytics Using Python with Machine Learning, Deep Learning and NLP Examples*, Apress (2018) ISBN-13 (electronic): 978-1-4842-3450-1
6. **Saurabh Chandrakar, Dr. Nilesh Bhaskarrao Bahadure** - *Python for Everyone, Learn and polish your coding skills in Python*, BPB Publications (2023), ISBN 978-93-5551-815-6
7. **Deepali R Vora, Gresha S Bhatia** - *Python Machine Learning Projects, Learn how to build Machine Learning projects from scratch*, BPB Publications (2023), ISBN 978-93-89898-27-9
8. **Trevor Hastie, Robert Tibshirani, Jerome Friedman,** *The Elements of*
9. *Statistical Learning - Data Mining, Inference, and Prediction 2nd ed.,* Springer, ISBN 9780387848570 https://hastie.su.domains/ElemStatLearn/ (free download)